Preference-Aware Deep Reinforcement Learning with GP-Evolved Actions for Dynamic Workflow Scheduling in Cloud–Edge Computing

Anonymous Authors

***Abstract*—Online workflow scheduling in heterogeneous Cloud–Edge systems must allocate resources before future workflow arrivals are known. Resource choices have state-dependent effects on both workflow tardiness and energy consumption. Edge Nodes avoid Cloud provisioning and reduce wide-area communication delays but have limited capacity, whereas Cloud Servers provide elastic and powerful computing capacity but incur additional provisioning and communication overheads. Their relative benefits further depend on task characteristics, workflow dependencies, and energy efficiency. Therefore, scheduling policies must adapt to user-requested objective trade-offs and evolving system states. Genetic programming (GP) hyper-heuristics have demonstrated effectiveness in evolving resource-selection rules for dynamic workflow scheduling, but usually apply them as fixed policies. Deep reinforcement learning enables adaptive control, yet directly treating a variable set of feasible resources as actions while handling multiple preferences complicates policy learning. We propose Preference-Aware Risk-Stratified Genetic Programming-assisted Deep Reinforcement Learning (PRS-GPDRL). Cooperative coevolution genetic programming (CCGP) first evolves coupled task- and resource-selection rules. Quality- and behavior-aware filtering then retains a compact resource-selection rule pool from the final CCGP archive. Each retained rule is paired with base and risk modes. A multi-objective double deep Q-network then selects a rule-mode action under the requested preference at each allocation decision. Across 12 scenarios (30 independent runs each), PRS-GPDRL achieves the best average ranks for hypervolume (HV) and inverted generational distance (IGD), with ranks of 1.406 and 1.478, respectively, and statistically outperforms or matches CCGP in 11 scenarios per metric. Gains are clearest on small- and medium-scale sets, and selected large-scale scenarios show additional HV gains. Behavioral and ablation evidence supports preference-dependent rule selection and risk-stratified adjustment.**

***Index Terms*—Cloud–Edge computing, dynamic workflow scheduling, genetic programming, deep reinforcement learning, multi-objective optimization.**

I. Introduction

CLOUD–EDGE computing provides a heterogeneous execution environment for workflow applications by combining latency-proximate Edge Nodes with elastic Cloud Servers. These applications include scientific workflows, data analytics pipelines, Internet of Things services, and emerging artificial intelligence agent orchestration services [1], [2], [3], [4]. Many workflow applications are represented as directed acyclic graphs (DAGs), where tasks are constrained by precedence and data dependencies [1], [2], [3]. As workflows arrive dynamically, a scheduler must order ready tasks and allocate heterogeneous resources without knowing future arrivals. Energy consumption has also become an important system-level constraint. Data centers consumed around 415 TWh worldwide in 2024, accounting for about 1.5% of global electricity demand, and their share of metered electricity consumption reached 23% in Ireland in 2025 [5], [6]. These trends make dynamic workflow scheduling in Cloud–Edge computing a key problem for improving quality of service while controlling resource use and energy consumption.

The main difficulty is that each resource allocation must be evaluated in the current workflow and resource state, because its effects may propagate beyond the task being assigned. *First*, workflow dependencies make task completion times affect downstream execution. The finish time of a task determines when its successors can be released and may further affect workflow completion and service quality. *Second*, Cloud–Edge heterogeneity creates different trade-offs between latency and capacity. Edge Nodes can shorten data paths and avoid Cloud provisioning, but their computing capacity is limited. Cloud Servers provide elastic capacity, but may introduce additional communication and provisioning overheads. *Third*, the CPU-sharing model considered in this paper couples the progress of tasks running on the same resource. Container virtualization provides a practical execution substrate for workflow tasks [7], and modern platforms support runtime resource adjustment through mechanisms such as Kubernetes in-place resource resizing and Docker CPU share or quota updates [8], [9], [10]. In our model, multiple tasks may execute on one resource and share its CPU capacity. Assigning a new task changes the running-task set, so the CPU allocated to CPU-sensitive tasks may be recomputed. As a result, the allocation may change remaining execution times, successor release times, resource active duration, and energy consumption. Therefore, an effective scheduler should consider dependency-driven task urgency, heterogeneous resource effects, and runtime CPU allocation when making online decisions. In a multi-objective setting, resource selection should also adapt to the requested tardiness–energy preference.

Many dynamic workflow scheduling studies use manual rules or genetic programming (GP) hyper-heuristics to support online decisions. Manual rules often combine task priorities, sub-deadlines, resource reuse, cost or energy estimates, and candidate filtering to guide task sequencing and resource allocation [11], [12], [13], [14], [15]. Their scheduling logic is explicit, but their performance depends on expert knowledge in selecting and combining task, workflow, and resource

attributes. When the execution environment, resource model, workload pattern, or objective preference changes, these rules often require redesign or careful adjustment. GP hyper-heuristics reduce this manual design effort by automatically evolving priority rules from selected attributes. Instead of searching for one schedule for one instance, GP searches the heuristic space and produces rules that can be applied to new dynamic instances. GP has been used to evolve single-tree, multi-objective, multi-tree, dual-tree, and cooperative rules for task selection, resource selection, and related workflow scheduling decisions [16], [17], [18], [19], [20], [21]. However, most GP hyper-heuristic methods deploy one evolved heuristic or a fixed set of cooperating rules throughout an online scheduling run. They do not decide online which evolved resource-selection rule is more suitable for the current task urgency, resource availability, CPU allocation, and objective preference.

Deep reinforcement learning (DRL) offers a way to learn online selection policies from scheduling experience, but two issues must be addressed in this setting. *First*, directly using feasible resources as actions is difficult for dynamic workflow scheduling in Cloud–Edge computing because the action set changes with task requirements, resource availability, and on-demand Cloud provisioning [22], [23], [24]. Rule-assisted DRL avoids this issue by using scheduling rules as fixed actions, where the selected rule ranks feasible candidates at the current decision point. Existing rule-assisted studies use manual rules or GP-evolved rules in dynamic scheduling [25], [26], [27], and GP-evolved actions have also been applied to dynamic Fog workflow scheduling [28]. However, these studies mainly address single-objective or fixed-objective settings, and rule sets based on manual rules still depend on expert design. *Second*, different users or scenarios may require different tardiness–energy trade-offs. Multi-objective reinforcement learning (MORL) and Pareto set learning can condition decisions on preferences [29], [30], [31], [32], [33], [34]. However, they do not provide a rule-mode action design that combines a compact pool of GP-evolved resource-selection rules with preference-aware online selection. This gap motivates combining GP-based generation of complementary resource-selection rules with DRL-based preference-aware online selection.

To address this gap, this paper proposes Preference-Aware Risk-Stratified Genetic Programming-assisted Deep Reinforcement Learning (PRS-GPDRL). Cooperative coevolution genetic programming (CCGP) first evolves coupled task-selection and resource-selection rules, so each resource-selection rule is evaluated together with a cooperating task-selection rule. A quality- and behavior-aware extraction procedure then constructs a compact pool of complementary resource-selection rules from the final CCGP archive. Each retained resource-selection rule is paired with a base mode and a risk mode. The base mode follows the selected GP rule, while the risk mode adds a bounded preference-aware local correction. Pairing the retained rules with these two modes gives a fixed rule-mode action space for the DRL agent. A multi-objective double deep Q-network (DDQN) learns to select rule-mode actions according to the current system state and requested preference. Fixed upward-rank ordering is used for ready-task sequencing, so the learned policy can focus on resource selection.

The main contributions are as follows:

- We establish a CCGP-to-DRL knowledge interface that extracts high-quality and behaviorally complementary resource-selection rules from a CCGP archive. Pairing the compact rule pool with base and risk modes converts these executable heuristics into a fixed rule-mode action space that ranks the variable feasible-resource set at each allocation decision.
- We develop two-level adaptive resource selection through risk-stratified action augmentation. A selected GP rule supplies the global ranking direction, while the risk mode conditionally applies a bounded local correction using the current state and requested preference. Feature-wise linear modulation (FiLM) conditioning, vector-valued action costs, and augmented weighted Tchebycheff scalarization enable one DDQN to serve different tardiness–energy preferences.
- Across 12 dynamic scheduling scenarios, with 30 independent runs conducted per scenario, PRS-GPDRL obtains the best average ranks for hypervolume (HV) and inverted generational distance (IGD), with ranks of 1.406 and 1.478, respectively. It is statistically better than or comparable to CCGP in 11 of 12 scenarios and to GATES and LWFF in at least 11 of 12 scenarios for each metric. The clearest advantages occur on small- and medium-scale workflow sets, with additional HV gains in selected large-scale scenarios.
- Behavioral analyses show preference-aligned changes at the outcome, rule, and mode levels: objective values respond to the requested preference, online selection migrates between energy- and tardiness-oriented rule roles, and local risk correction is activated selectively. Ablation and diagnostic results further distinguish the contributions of complementary rule coverage, preference-dependent representation, local correction, and task ordering.

II. Related Work

A. Rule-Based Dynamic Workflow Scheduling

Dynamic workflow scheduling in cloud, fog, and Cloud–Edge environments has been studied under deadline, cost, energy, and utilization objectives [11], [12], [13], [14], [15]. Manual rules and constructive methods remain important because they can be directly applied during online scheduling. Fan et al. [11] proposed Online Hybrid Dynamic Scheduling (OHDS), which integrates task merging, sub-deadline assignment, hybrid scheduling of multiple workflows, dynamic priority adjustment, and virtual machine migration based on CPU utilization. Sun et al. [12] proposed Real-time Multi-workflow Energy-efficient Scheduling (RMES) for container-based clouds, where sub-deadlines, container-machine affinity, resource reuse, and rescheduling of remaining tasks are used for energy-efficient real-time scheduling. Fard et al. [15] proposed Least Waste Fast First (LWFF) for microservice

placement, using CPU and memory requests to improve multi-resource utilization and throughput. These methods provide clear scheduling mechanisms for their target resource models and objectives. Although some of them update priorities or reschedule tasks during execution, the priority and scoring rules are still specified by design. Therefore, they provide limited support for changing the scheduling behavior online when workload conditions or objective preferences vary.

Learning-based methods learn scheduling policies or candidate evaluations from data or simulation experience. DRL and multi-agent DRL have been used to address time–energy scheduling, multi-objective resource allocation, and collaborative workflow offloading in Cloud–Edge environments [22], [23], [35]. GATES uses graph attention to encode variable ready-task and virtual-machine states, and trains the allocation policy through an evolution strategy [24]. Reinforcement learning-based hyper-heuristics have also been studied for energy-efficient cloud workflow scheduling [36]. These studies improve adaptive decision making for dynamic workflow scheduling, but their learned components mainly produce direct allocation policies, resource scores, or selections among predefined low-level heuristics. They do not focus on online deployment of a pool of evolved resource-selection rules.

GP hyper-heuristics address rule generation by searching for executable priority rules. Escott et al. [16] introduced GP hyper-heuristics for dynamic cloud workflow scheduling to generate rules from a richer set of task and resource attributes. Yu et al. [17] extended this idea to multi-objective dynamic workflow scheduling. Xu et al. [18], Sun et al. [19], and Yang et al. [20] further studied GP rule generation for fog, multi-cloud, and cloud workflow scheduling with different rule representations. Sun et al. [21] proposed CCGP for dynamic joint workflow scheduling and container scaling in Cloud–Fog computing. CCGP evolves task-selection, resource-selection, and container-scaling rules in cooperating subpopulations. These studies show that GP can automatically discover scheduling rules for dynamic workflow environments. However, the evolved heuristic or cooperating rule set is normally fixed before online execution. Thus, GP can provide useful rules, but it does not decide online which resource-selection rule should be applied under the current workflow state and objective preference.

Rule-assisted DRL addresses online rule selection by defining actions over scheduling rules rather than over a changing set of candidate jobs or resources. Liu et al. [25] used manual dispatching rules as actions for DRL in dynamic flexible job-shop scheduling. This indirect action representation avoids direct action definitions over variable candidate sets, but its performance is limited by the manual rule library. Xu et al. [26] replaced manual actions with heuristics evolved by niching GP, and the behaviorally different heuristics are then used as DRL actions. Zhao et al. [27] studied GP-assisted DRL for dynamic flexible job-shop scheduling, and He et al. [28] applied GP-evolved actions to dynamic fog workflow scheduling. These studies are closely related to the rule-action design in this paper. However, they mainly address dynamic job-shop scheduling or fixed-objective workflow scheduling. They do not directly consider preference-aware online selection from a compact resource-selection rule pool extracted from a multi-objective GP archive for dynamic workflow scheduling in Cloud–Edge computing.

B. Preference-Aware Multi-Objective Learning

MORL studies sequential decision making with vector-valued returns and different objective trade-offs [29], [30]. One line learns multiple policies or a continuous Pareto manifold to cover different non-dominated regions [37], [38]. Another line conditions a shared value function or policy on the requested preference. For example, Abels et al. [31] conditioned deep Q-networks on dynamic objective weights, and Yang et al. [32] learned a generalized policy for different preferences. Recent MORL studies further improve preference control and adaptation in learned policies [39], [40], [41]. These methods provide useful preference representations for multi-objective decision making. However, their focus is preference-aware policy learning rather than scheduling-rule generation and online rule deployment.

Preference-aware ideas have also been introduced into evolutionary multi-objective optimization. Pareto set learning maps a preference vector to an approximate Pareto solution through scalarization-based training [33]. Xu et al. [34] proposed Pareto Set Learning GP (PSLGP) for multi-objective dynamic scheduling. PSLGP uses the preference as an input to a single GP heuristic, reducing the need to manage many heuristics for different Pareto regions. This line shows that preferences can guide evolved decision rules. The remaining issue for this paper is different: how to use DRL to select online among multiple GP-evolved resource-selection rules while the feasible resource set and workflow state change over time. This motivates preference-aware online deployment of complementary GP-evolved resource-selection rules for dynamic workflow scheduling in Cloud–Edge computing.

III. Problem Modeling

We consider a heterogeneous Cloud–Edge computing environment for dynamic workflow execution. Let $R = C \cup E$ be the set of computing resources, where C and E denote the sets of Cloud Servers and Edge Nodes, respectively. Each resource $r \in R$ is modeled as a virtual machine (VM) with CPU capacity $\Gamma^c(r)$. Edge Nodes form a fixed resource pool with limited capacity, whereas Cloud Servers can be provisioned on demand. Tasks are packaged as containers and deployed to selected resources. A resource can run multiple containers simultaneously, but the total CPU allocated to the running containers cannot exceed the CPU capacity of the resource at any moment. A container is the minimum execution unit and executes one task at a time. The scheduling model treats CPU capacity as the adjustable resource. Let B_{edge} and L_{edge} be the bandwidth and latency for communications within the Edge tier, respectively. Let B_{cloud} and L_{cloud} be the bandwidth and latency for communications involving Cloud Servers, respectively. Each resource has static power and CPU-based dynamic power. Static power $p^s(r)$ consists of base power $p^b(r)$ and CPU-based idle power $p^{ic}(r)$, where $p^s(r) = p^b(r) + p^{ic}(r)$. Dynamic power is defined as the difference between CPU full

power and CPU idle power, namely $p^{dc}(r) = p^{fc}(r) - p^{ic}(r)$. The energy consumption of both Cloud Servers and Edge Nodes is considered in this work.

Let $\mathcal{W}$ be the set of workflow applications. Each workflow $\omega \in \mathcal{W}$ contains a task set $A(\omega)$. The arrival time, weight, and deadline of workflow ω are $\rho^a(\omega)$, $\rho^w(\omega)$, and $\rho^d(\omega)$, respectively. Each task $\alpha \in A(\omega)$ is represented by $\{\phi(\alpha), \tau^l(\alpha), \mu^c(\alpha), A^{pre}(\alpha), A^{suc}(\alpha)\}$, where $\phi(\alpha) \in \{0, 1\}$ denotes the task category, $\tau^l(\alpha)$ is its CPU workload, $\mu^c(\alpha)$ is its minimum CPU requirement, and $A^{pre}(\alpha), A^{suc}(\alpha) \subseteq A(\omega)$ are its predecessor and successor task sets, respectively. Let $d(\alpha_1, \alpha_2)$ be the data transferred from task α_1 to its successor task $\alpha_2 \in A^{suc}(\alpha_1)$. The execution time of a task depends on its category and allocated CPU capacity. Tasks are divided into two categories.

- $\phi(\alpha) = 0$. The allocated CPU must meet the task's minimum CPU requirement, but allocating more CPU does not reduce its execution time. Such tasks are CPU-insensitive.
- $\phi(\alpha) = 1$. The allocated CPU must meet the task's minimum CPU requirement, and a larger allocation can reduce its execution time. Such tasks are CPU-sensitive.

We formulate dynamic workflow scheduling with deadline constraints as a bi-objective optimization problem. The objectives are to minimize system-oriented total energy consumption and workflow-oriented total weighted tardiness under the following scheduling and resource constraints:

- No task can be executed until all its predecessor tasks have been completed.
- Each task must be executed by a container whose allocated CPU satisfies the task's minimum CPU requirement.
- Each resource can deploy and run multiple containers simultaneously, but the sum of their allocated CPU capacities cannot exceed the resource's CPU capacity at any moment.

We model dynamic workflow scheduling as a discrete-event control process. Let $\mathcal{T}$ be the set of decision points, and let $t \in \mathcal{T}$ be a specific decision point. Decision points typically correspond to moments when the system state changes, such as when a workflow is submitted, a task becomes ready, a task starts execution, or a task completes. Let $x^s(\alpha)$ and $x^f(\alpha)$ be the execution start time and execution finish time of task α, respectively. Let $y(\alpha) \in R$ be the assigned resource of task α in the schedule. Let $\nu^c(\alpha, t)$ be the CPU allocated to the container executing task α at time t.

The problem can then be formulated as

$$\min \mathbb{E} = \sum_{r \in R} \int_{\mathcal{T}} P(r, t) dt, \tag{1}$$

$$\min \mathbb{T} = \sum_{\omega \in \mathcal{W}} \rho^w(\omega) \times \max \left\{ 0, \max_{\alpha \in A(\omega)} \left\{ x^f(\alpha) \right\} - \rho^d(\omega) \right\}, \tag{2}$$

$$s.t. : x^s(\alpha) \geq \rho^a(\omega), \ \forall \ \omega \in \mathcal{W}, \alpha \in A(\omega), \tag{3}$$

$$x^f(\alpha_1) + x^c(\alpha_1, \alpha_2) + x^l(\alpha_2) \leq x^s(\alpha_2), \ \forall \alpha_2 \in A^{suc}(\alpha_1), \tag{4}$$

$$\begin{cases} x^f(\alpha) - x^s(\alpha) = \dfrac{\tau^l(\alpha)}{\mu^c(\alpha)}, & \phi(\alpha) = 0, \\ \displaystyle\int_{x^s(\alpha)}^{x^f(\alpha)} \nu^c(\alpha, t) dt = \tau^l(\alpha), & \phi(\alpha) = 1. \end{cases} \tag{5}$$

$$\nu^c(\alpha, t) \geq \mu^c(\alpha), \ \forall t \in \mathcal{T}, \tag{6}$$

$$\sum_{\alpha \in A_r(t)} \nu^c(\alpha, t) \leq \Gamma^c(r), \ \forall r \in R, \forall t \in \mathcal{T}, \tag{7}$$

$$x^l(\alpha) = \begin{cases} L_{\text{edge}}, & y(\alpha) \in E, \\ L_{\text{cloud}}, & y(\alpha) \in C. \end{cases} \tag{8}$$

$$x^c(\alpha_1, \alpha_2) = \begin{cases} 0, & y(\alpha_1) = y(\alpha_2), \\ \dfrac{d(\alpha_1, \alpha_2)}{B_{\text{edge}}}, & y(\alpha_1) \neq y(\alpha_2), \\ & y(\alpha_1), y(\alpha_2) \in E, \\ \dfrac{d(\alpha_1, \alpha_2)}{B_{\text{cloud}}}, & \text{otherwise.} \end{cases} \tag{9}$$

The CPU allocation of containers is updated whenever a task starts or completes on a resource. Given the assigned resources $y(\alpha)$, the CPU allocation at time t is determined by the running task set on each resource. Let $A_r(t)$ be the set of running tasks on resource r at time t, i.e.,

$$A_r(t) = \{\alpha \mid y(\alpha) = r, \ x^s(\alpha) \leq t < x^f(\alpha)\}. \tag{10}$$

The remaining CPU capacity after satisfying the minimum CPU requirements of all running tasks is

$$\Gamma^c_{\text{rem}}(r, t) = \Gamma^c(r) - \sum\nolimits_{\beta \in A_r(t)} \mu^c(\beta). \tag{11}$$

Let $S_r(t) = \{\alpha \in A_r(t) \mid \phi(\alpha) = 1\}$ be the set of CPU-sensitive running tasks on resource r at time t. For each running task $\alpha \in A_r(t)$, the CPU allocation is calculated as

$$\nu^c(\alpha, t) = \begin{cases} \mu^c(\alpha), & \phi(\alpha) = 0, \\ \mu^c(\alpha) + \dfrac{\Gamma^c_{\text{rem}}(r, t)}{|S_r(t)|}, & \phi(\alpha) = 1. \end{cases} \tag{12}$$

Thus, CPU-insensitive tasks use their minimum CPU requirements, while CPU-sensitive tasks equally share the remaining CPU capacity. The CPU utilization of resource r at time t is

$$U^c(r, t) = \frac{\sum_{\alpha \in A_r(t)} \nu^c(\alpha, t)}{\Gamma^c(r)}. \tag{13}$$

The instantaneous power consumption of resource r is modeled as

$$P(r, t) = p^s(r) + p^{dc}(r) \left(U^c(r, t)\right)^3. \tag{14}$$

The above equations hold for $\forall \ \omega \in \mathcal{W}$, $\forall \ \alpha, \alpha_1, \alpha_2 \in A(\omega)$, $\forall \ r \in R$, and $\forall t \in \mathcal{T}$ when the corresponding quantities are defined. Eqs. (1) and (2) are the two objectives considered in this work. Eq. (1) gives the total energy consumption by integrating the instantaneous power $P(r, t)$ over all Cloud Servers and Edge Nodes. Eq. (2) gives the workflow-oriented total weighted tardiness. Constraint (3) ensures that a workflow cannot be executed until it is submitted, and thus the scheduler has no information about that workflow before its arrival. Constraint (4) enforces workflow precedence and data transfer dependencies. A task can start only after all of its immediate predecessors have finished and their output data have arrived.

Eq. (5) defines the execution time of the two task categories, ensuring that each task α completes workload $\tau^l(\alpha)$ during $[x^s(\alpha), x^f(\alpha)]$ under the corresponding CPU allocation. Constraint (6) ensures that the CPU allocation of each running task satisfies its minimum CPU requirement. Constraint (7) ensures that the total CPU allocated to running tasks on a resource does not exceed the resource's CPU capacity at time t. Eq. (8) defines the access latency $x^l(\alpha)$ according to whether task α is assigned to an Edge Node or a Cloud Server. Eq. (9) gives the communication time between task α_1 and its successor α_2 according to their assigned resources. Eqs. (10)-(12) define the elastic CPU sharing rule. Eq. (10) gives the set of running tasks on resource r. Eq. (11) calculates the remaining CPU capacity after the minimum CPU requirements of all running tasks have been satisfied. Eq. (12) then assigns the minimum CPU requirement to each CPU-insensitive task and equally divides the remaining CPU capacity among CPU-sensitive tasks. Eq. (13) gives the CPU utilization of resource r at time t. Eq. (14) models the instantaneous power consumption of resource r, which consists of static power $p^s(r)$ and CPU-based dynamic power that scales cubically with CPU utilization. For on-demand Cloud Servers, the power model is evaluated only over recorded utilization intervals of provisioned resources. Unprovisioned Cloud Servers do not contribute to the energy integral in Eq. (1).

IV. The Proposed PRS-GPDRL Algorithm

A. Overall Framework of PRS-GPDRL

Fig. 1 shows the overall framework of PRS-GPDRL, which consists of offline training and online scheduling. For each training scenario, CCGP first produces an archive of complete scheduling heuristics, each of which contains a task-selection rule and a resource-selection rule. Although both rules are evolved cooperatively, PRS-GPDRL uses the resource-selection rules to construct the DDQN action space and uses upward-rank ordering for ready-task sequencing. The task-ordering ablation experiment in Section V-F supports the use of upward-rank ordering by comparing it with the GP task-selection rule while the resource-selection rule pool is fixed. The results show that upward-rank ordering is statistically comparable to the GP task-selection rule in 11 of 12 scenarios. Including task-selection rules would also introduce many additional action combinations, increasing training cost and making convergence more difficult. Therefore, PRS-GPDRL uses fixed upward-rank ordering for ready-task sequencing and focuses the learned action space on resource selection.

The offline stage extracts a compact resource-selection rule pool $\mathcal{G}$ from the final CCGP archive. Each retained rule is paired with the base and risk modes in $\mathcal{M}$, and the resulting rule-mode action space $\mathcal{A}$ remains fixed while the feasible resource set varies at runtime. A multi-objective DDQN is trained to select actions from $\mathcal{A}$ according to the task attributes, feasible resource attributes, and preference weights.

In online scheduling, fixed upward-rank ordering is used only when multiple tasks are ready at the same decision point [42]. For the selected ready task, the trained DDQN chooses a resource-selection rule and a mode according to the current state and requested preference. The selected rule-mode action then scores the feasible resources, and the task is assigned to the resource with the minimum priority score.

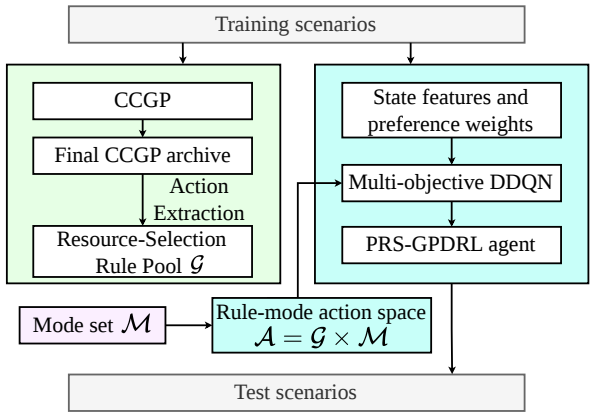


Fig. 1. Overall framework of PRS-GPDRL.

B. CCGP Training and Action Extraction

CCGP training. The CCGP stage follows the cooperative coevolution mechanism used for tree-based scheduling rules. It maintains two subpopulations, one for task-selection rules and the other for resource-selection rules, so that the two coupled decisions can be evolved cooperatively. During evaluation, an individual from one subpopulation is paired with a representative from the other subpopulation to form a complete scheduling heuristic. The complete heuristic is evaluated by energy consumption and weighted tardiness, and the evaluated heuristics are recorded in the archive set (AS). Until the stopping criterion is met, CCGP updates the two subpopulations by elitism, reproduction, crossover, and mutation. At the final generation, duplicate solutions are removed from AS, and the first non-dominated front is retained as the source of resource-selection rules for PRS-GPDRL.

Rule representation. Each tree-based rule is a priority expression composed of terminal and function nodes. Terminal nodes provide scheduling attributes of the current workflow, ready task, and feasible resource, while function nodes combine terminal values to calculate priority scores. The task-selection and resource-selection rules use different terminal subsets and share the same function set. Table I summarizes the terminals used by the CCGP rules and the corresponding attributes used by the DDQN state encoder. The state encoder uses these attributes so that the DDQN observes information consistent with the GP rule inputs. The GP and DDQN columns indicate how each attribute is used by the two components. Further details of CCGP training and the computation of GP terminal values are given in [21].

Action extraction. After CCGP training, PRS-GPDRL constructs $\mathcal{G}$ from the first non-dominated front retained from AS. For a complete heuristic i in this front, let g_i denote its resource-selection rule. Different heuristics may contain resource-selection rules that make the same choices at representative decision points. Therefore, the extraction procedure

TABLE I
TERMINAL, FUNCTION, AND STATE-ENCODER ATTRIBUTES USED BY PRS-GPDRL.

Notation	Description	GP	DDQN
CAT	Category of a task.	T	T
SD	Sub-deadline of a task.	T	T
ET	Execution time of a task on a reference resource.	T	T
CPUA	Minimum CPU requirement of a task.	T	T
WT	Weight of a workflow.	T	T
UR	Task upward-rank value.	T	T
NSA	Number of direct successors of a task.	T	T
NPA	Number of direct predecessors of a task.	T	–
NKQ	Number of tasks in the ready queue.	T	–
TETQ	Total ET of tasks in the ready queue.	T	–
NRA	Number of remaining tasks in a workflow.	T	T
TETRA	Total ET of remaining workflow tasks.	T	T
VCT	Average communication time for a task.	T	T
NEW	Whether the resource is newly created.	–	R
WAIT	Waiting time before the task starts on a resource.	R	R
ETAR	Minimum execution time of a task on a resource.	R	R
CRCPU	Current remaining CPU of a resource.	R	R
ST	Slack time of a task on a resource.	R	R
STPR	Static power of a resource.	R	R
DPR	Dynamic power of a resource.	R	R
NTR	Number of tasks on a resource.	R	R
NKW	Number of scheduled tasks from the same workflow in the Cloud or Edge tier.	R	R
CPUR	CPU capacity of a resource.	R	R
CTR	Communication time of a task on a resource.	R	R
w_T	Preference weight for weighted tardiness.	–	P
w_E	Preference weight for energy consumption.	–	P
Function set $+$, $-$, $\times$, protected /, Max, and Min.		T/R	–

In the GP column, T and R denote task- and resource-selection rule terminals. In the DDQN column, T, R, and P denote task, feasible resource, and preference attributes, respectively.

retains rules from different regions of the validation trade-off front and removes behaviorally duplicate rules. It consists of validation evaluation, decision point sampling, behavioral representation, and anchor selection.

Validation evaluation. Each complete heuristic in the retained front is evaluated on a validation set of random seeds to obtain a bi-objective performance estimate for anchor selection. The validation performance is represented by the mean energy consumption and mean weighted tardiness over three independent runs. Let $S_i^{\mathrm{val}} = \widehat{\mathbb{E}}_i^{\mathrm{val}} + \widehat{\mathbb{T}}_i^{\mathrm{val}}$ denote the normalized validation score of heuristic i, where $\widehat{\mathbb{E}}_i^{\mathrm{val}}$ and $\widehat{\mathbb{T}}_i^{\mathrm{val}}$ are its min–max normalized validation energy and weighted tardiness, respectively. The score is used only for validation-based selection among CCGP heuristics and does not replace the final test evaluation.

Decision point sampling. A reference scheduling heuristic is selected from the validation-evaluated heuristics by minimizing S_i^{val}. The reference heuristic is then executed to collect representative resource allocation decision points. At each sampled decision point, all feasible resources are recorded with their current terminal attributes. These samples provide a common set of online decision contexts for comparing the behavior of different resource-selection rules.

Behavioral representation. The behavior of resource-selection rule g_i is measured by replaying it on the sampled decision points. At decision point d, the rule scores all feasible resources and selects the resource with the minimum priority score. The behavior vector of g_i is

$$\mathbf{b}_i = (\pi_i(d_1), \pi_i(d_2), \ldots, \pi_i(d_M)), \tag{15}$$

where $\pi_i(d_m)$ is the resource index selected by g_i at sampled decision point d_m, and $M = 20$ is the number of sampled decision points. In implementation, $\pi_i(d_m)$ is encoded as a resource-selection indicator vector, so tied best resources are represented as the same selected set. Rules producing identical behavior vectors are grouped together, and the rule whose complete heuristic has the lowest S_i^{val} is retained as the representative.

Anchor selection. Let K denote the maximum requested size of the resource-selection rule pool. If the number of behaviorally unique rules does not exceed K, all representatives are retained. For the default $K = 5$, the retained rules are otherwise selected from five representative positions on the validation front: the minimum-energy endpoint, a low-energy neighbor, a balanced anchor, a low-tardiness neighbor, and the minimum-tardiness endpoint. Selecting these positions preserves rules associated with different tardiness–energy trade-off regions, so that the DDQN can choose among complementary resource-selection behaviors during online scheduling. The endpoints minimize $\widehat{\mathbb{E}}_i^{\mathrm{val}}$ and $\widehat{\mathbb{T}}_i^{\mathrm{val}}$, respectively. After excluding the endpoints, each neighbor is drawn from the corresponding 5% normalized objective band. A heuristic belongs to this band when its corresponding normalized objective value does not exceed 0.05. If the band contains too few behaviorally distinct rules, top-ranked heuristics under the same objective are also considered. Let i_E and i_T denote the two endpoint indices. The balanced anchor is selected by

$$\begin{aligned} S_i^{\mathrm{bal}} &= \max\left\{\widehat{\mathbb{E}}_i^{\mathrm{val}}, \widehat{\mathbb{T}}_i^{\mathrm{val}}\right\} + 0.05\left(\widehat{\mathbb{E}}_i^{\mathrm{val}} + \widehat{\mathbb{T}}_i^{\mathrm{val}}\right), \\ i_{\mathrm{bal}} &= \arg\min_{i \notin \{i_E, i_T\}} S_i^{\mathrm{bal}}. \end{aligned} \tag{16}$$

For the $K = 3$ ablation, the minimum-energy endpoint, the balanced anchor, and the minimum-tardiness endpoint are retained. If a selected rule duplicates an already retained behavior, the next behaviorally distinct rule from the same region or objective ranking is used.

For a given CCGP run b, the retained resource-selection rules form $\mathcal{G} = \{g_1, \ldots, g_{K_b}\}$, where $K_b \le K$ is the actual number retained. Combining $\mathcal{G}$ with the two modes gives the rule-mode action space $\mathcal{A}$ used by the DDQN.

C. Risk-Stratified Action Augmentation

Each extracted resource-selection rule defines a priority function for ranking the feasible resources. The normalized GP score preserves the ranking behavior learned by CCGP, but the score itself is not explicitly adjusted by the requested preference. Therefore, PRS-GPDRL pairs each resource-selection rule with a *base* mode and a *risk* mode. The base mode uses the normalized GP score directly. The risk mode keeps the GP score as the primary ranking term and adds a bounded correction derived from the local tardiness-related and energy-related costs defined below. In this way, the DDQN can choose either the original GP ranking or its preference-aware local correction for the current decision.

Let $\mathbf{w} = (w_T, w_E)$ be the current preference vector, where w_T and w_E are the weights for weighted tardiness and energy consumption, respectively, and $w_T + w_E = 1$. At decision point t, let $\mathcal{R}_t$ be the feasible resource set and let $n = |\mathcal{R}_t|$. For a selected resource-selection rule $g_i \in \mathcal{G}$, let $\hat{g}_i(r)$ denote its min–max normalized priority score for resource $r \in \mathcal{R}_t$. For all min–max normalizations in this work, if all resources have the same value, the corresponding normalized score is set to zero for all resources.

PRS-GPDRL estimates two local costs for each resource in the same feasible resource set. These costs are used only to stratify feasible resources under the requested preference, rather than to replace the selected GP rule. For the tardiness-related local cost, let α_t be the current ready task at decision time t, let $\mathsf{SD}(\alpha_t)$ be its sub-deadline, and let ω be the workflow containing α_t. For resource r, let $t^{\mathrm{ava}}_{\alpha_t,r}$ be the actual available time for assigning α_t to r. To evaluate this feasible assignment, α_t is temporarily assigned to r at $t^{\mathrm{ava}}_{\alpha_t,r}$, and CPU allocation is updated according to Eq. (12). Let $\hat{x}^f_{\alpha_t,r}$ be the estimated finish time of α_t under this temporary assignment. The local cost combines the estimated time from the current decision point to task completion with the workflow-weighted sub-deadline violation:

$$c_T(r) = \hat{x}^f_{\alpha_t,r} - t + \rho^w(\omega)\max\{0, \hat{x}^f_{\alpha_t,r} - \mathsf{SD}(\alpha_t)\}, \quad (17)$$

For the energy-related local cost, PRS-GPDRL compares the resource state before and after the tentative assignment. Let U^0_r and U^1_r be the CPU utilization before and after assignment, and let H^0_r and H^1_r be the corresponding busy horizons measured from $t^{\mathrm{ava}}_{\alpha_t,r}$ to the tail completion time of the resource. Using the power terms defined in Section III, the dynamic energy increment is

$$\Delta E^{\mathrm{dyn}}_r = p^{dc}(r)\left((U^1_r)^3 H^1_r - (U^0_r)^3 H^0_r\right). \quad (18)$$

$\Delta E^{\mathrm{dyn}}_r$ estimates the dynamic energy increment caused by assigning the current task to resource r. The static energy increment is calculated according to whether r is newly created, already active, or idle:

$$\Delta E^{\mathrm{sta}}_r = \begin{cases} p^s(r) H^1_r, & \text{new}, \\ p^s(r)\left(t^{\mathrm{tail},1}_r - t^{\mathrm{tail},0}_r\right), & \text{active}, \\ p^s(r)\left(t^{\mathrm{ava}}_{\alpha_t,r} - t^{\mathrm{last}}_r + H^1_r\right), & \text{idle}, \end{cases} \quad (19)$$

where $t^{\mathrm{tail},0}_r$ and $t^{\mathrm{tail},1}_r$ are the tail completion times before and after assignment, and t^{last}_r is the last utilization event time of r. For an idle resource, the static energy increment includes the idle interval since its last utilization event and the new busy horizon caused by assigning the current task. The energy-related local cost is

$$c_E(r) = \Delta E^{\mathrm{dyn}}_r + \Delta E^{\mathrm{sta}}_r. \quad (20)$$

Let $\hat{c}_T(r)$ and $\hat{c}_E(r)$ be the min–max normalized local costs used to make the two components comparable within the current feasible resource set. A preference-weighted auxiliary score is defined as

$$c_{\mathbf{w}}(r) = w_T \hat{c}_T(r) + w_E \hat{c}_E(r). \quad (21)$$

The auxiliary score ranks the feasible resources for risk stratification. Let $\mathrm{rank}_{\mathbf{w}}(r) \in \{1, \ldots, n\}$ be the ascending rank of $c_{\mathbf{w}}(r)$. PRS-GPDRL maps this rank to four risk strata:

$$\chi_{\mathbf{w}}(r) = \begin{cases} 0.00, & \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.10n \rceil, \\ 0.20, & \lceil 0.10n \rceil < \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.25n \rceil, \\ 0.50, & \lceil 0.25n \rceil < \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.50n \rceil, \\ 1.00, & \mathrm{rank}_{\mathbf{w}}(r) > \lceil 0.50n \rceil. \end{cases} \quad (22)$$

Because a smaller $c_{\mathbf{w}}(r)$ indicates a lower local cost under the requested preference, resources with better auxiliary ranks receive zero or small $\chi_{\mathbf{w}}(r)$ values, whereas resources with worse auxiliary ranks receive larger values. The risk mode uses these stratum values as bounded correction terms, so the local cost estimates can influence the final ranking without replacing the selected GP rule.

For the base mode, the risk term is disabled. For the risk mode, the mixing coefficient is

$$\beta_{\mathrm{risk}}(\mathbf{w}) = \beta_{\max}|w_T - w_E|^2, \quad (23)$$

where $|w_T - w_E|$ measures the departure from the balanced preference. When $w_T = w_E = 0.5$, the risk correction is disabled and the risk mode reduces to the base mode. The correction becomes stronger when the requested preference moves toward either objective endpoint and reaches $\beta_{\max}$ at an extreme preference.

The final priority score for resource r under resource-selection rule g_i and mode m is

$$p_m(r; g_i) = \begin{cases} \hat{g}_i(r), & m = \text{base}, \\ \hat{g}_i(r) + \beta_{\mathrm{risk}}(\mathbf{w})\chi_{\mathbf{w}}(r), & m = \text{risk}. \end{cases} \quad (24)$$

For a selected rule g^* and mode m^*, the selected resource is $r^*_t = \arg\min_{r \in \mathcal{R}_t} p_{m^*}(r; g^*)$. Therefore, the risk mode does not replace the selected GP rule. It only adds a bounded stratum correction to the normalized GP score. The final priority score retains the selected GP rule as its main component, while the auxiliary local estimates can change the ranking only within the range controlled by $\beta_{\mathrm{risk}}(\mathbf{w})$.

D. Markov Decision Process Formulation for Preference-Aware Rule Selection

After the current ready task has been selected, PRS-GPDRL determines which rule-mode action should be used to assign this task to a feasible resource. We model this resource allocation process as a Markov decision process (MDP) represented by $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbf{c}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the rule-mode action space, $\mathcal{P}$ is the state transition kernel, $\mathbf{c}$ is the vector cost, and γ is the discount factor.

At decision point t, state $s_t \in \mathcal{S}$ contains the attributes of the current ready task, the feasible resources, and the requested preference, as marked in the DDQN column of Table I. Task and preference attributes have fixed lengths, whereas the feasible resource set has a variable size. The feasible resource list is padded and accompanied by a validity mask, which allows the DDQN to observe the current feasible resources while keeping the action outputs defined over rule-mode pairs.

Let $\mathcal{M} = \{\text{base}, \text{risk}\}$ be the mode set. The action space is

$$\mathcal{A} = \mathcal{G} \times \mathcal{M}, \qquad a_t = (g_i, m), \qquad g_i \in \mathcal{G},\ m \in \mathcal{M}. \quad (25)$$

An actual pool size of K_b yields $2K_b$ rule-mode actions in run b. Executing a_t applies the selected resource-selection rule and mode to the feasible resources as defined in Section IV-C and assigns the current task to r_t^*. The scheduling process then advances to the next decision point or to episode termination, producing the next state and transition cost.

Each MDP transition supplies the two-dimensional immediate cost used to train the DDQN. Let ΔT_t and ΔE_t denote the tardiness-related cost and energy increment, respectively, produced by the simulator transition from one decision point to the next. The stored cost is

$$\mathbf{c}_t = (\Delta T_t, \kappa \Delta E_t), \quad (26)$$

where κ aligns the numerical scales of the two components during training. Evaluation uses the original objective values.

E. Preference-Aware Multi-Objective DDQN

PRS-GPDRL uses a multi-objective DDQN [43] to select rule-mode actions. At each decision point, the observation is encoded into a state representation and fed to a vector-valued DDQN. The DDQN estimates the cumulative cost of each rule-mode action, and the current preference is used to scalarize the two objective dimensions for both action selection and network update. Since the DDQN selects rule-mode actions rather than resources directly, the learned action space remains fixed, and the actual resource is selected by applying the chosen rule-mode action to the feasible resources observed at that decision point.

Fig. 2 illustrates the state encoder and rule-mode action selection. The raw observation s_t contains task features, feasible resource features, and preference weights. The state encoder maps s_t to an embedding $\mathbf{z}_t$, and the DDQN estimates vector action costs over $\mathcal{A}$. Augmented weighted Tchebycheff scalarization selects $a_t^* = (g^*, m^*)$, where $g^* \in \mathcal{G}$ and $m^* \in \mathcal{M}$ are the selected rule and mode. Algorithm 1 then applies the selected rule and mode to score the feasible resources and assign the ready task α_t to r_t^*.

Let $\mathbf{x}_t^{\text{task}}$, $\mathbf{X}_t^{\text{res}}$, and $\mathbf{w}$ denote the task features, feasible resource features, and preference vector, respectively. Continuous task and resource attributes are transformed using signed logarithmic scaling, while categorical indicators remain unchanged. The task encoder comprises two 64-unit layers, each followed by an exponential linear unit (ELU) activation. Feasible resources are separated into Edge Node and Cloud Server sets. Each tier-specific resource encoder contains two 64-unit ELU layers that project resource features into key representations, followed by a 64-dimensional linear value projection. Single-head masked scaled dot-product attention uses the task embedding as the query and the projected resource representations as keys to aggregate the corresponding values into a 64-dimensional tier-level representation. The task embedding and the two tier-level representations are concatenated and passed through two 128-unit fusion layers, with an ELU activation after the first, to produce $\mathbf{h}_{\text{fuse}} \in \mathbb{R}^{128}$.

A FiLM layer [44] modulates the fused representation according to the preference. A two-layer preference multilayer perceptron (MLP) maps the two preference weights through a 32-unit ELU hidden layer to a 256-dimensional output, which is split into 128-dimensional raw scale and shift vectors. The scale is parameterized as $\boldsymbol{\gamma}_{\text{film}}(\mathbf{w}) = 1 + 0.3\tanh(\widetilde{\boldsymbol{\gamma}}_{\text{film}}(\mathbf{w}))$, which bounds its elements between 0.7 and 1.3 and prevents multiplicative sign reversal or excessive amplification. The shift remains unconstrained. The FiLM transformation is

$$\mathbf{z}_t = \boldsymbol{\gamma}_{\text{film}}(\mathbf{w}) \odot \mathbf{h}_{\text{fuse}} + \boldsymbol{\delta}_{\text{film}}(\mathbf{w}), \quad (27)$$

where $\boldsymbol{\gamma}_{\text{film}}(\mathbf{w})$ and $\boldsymbol{\delta}_{\text{film}}(\mathbf{w})$ are preference-dependent scale and shift vectors. The resulting $\mathbf{z}_t \in \mathbb{R}^{128}$ is the DDQN state embedding.

The 128-dimensional state embedding is processed by a shared two-layer MLP with hidden sizes 256 and 128 and rectified linear unit (ReLU) activations. A linear output layer then produces a two-dimensional cost estimate for every rule-mode action:

$$\mathbf{Q}_\theta(s_t, a) = \left(Q_\theta^T(s_t, a), Q_\theta^E(s_t, a)\right), \quad a \in \mathcal{A}. \quad (28)$$

where Q_θ^T and Q_θ^E estimate cumulative tardiness-related and scaled energy costs, respectively. Keeping the two estimates separate allows the same network to evaluate the same action space under different preferences.

Because lower values are better for both objectives, action selection minimizes an augmented weighted Tchebycheff cost. For state s, the estimated ideal component for objective j is $q_\theta^{j,\min}(s) = \min_{a' \in \mathcal{A}} Q_\theta^j(s, a')$, where $j \in \{T, E\}$. The scalarized cost is

$$\begin{aligned}\psi_{\mathbf{w}}(\mathbf{Q}_\theta(s,a)) = &\max_{j\in\{T,E\}} w_j\Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s)\Big) \\ &+ \zeta \sum_{j\in\{T,E\}} w_j\Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s)\Big),\end{aligned} \quad (29)$$

where ζ is the augmentation coefficient. The estimated ideal point gives deviations from the best predicted action costs for each objective, and the augmentation term reduces ties. During training, Eq. (29) gives the preference-specific scalar objective used for the DDQN update. During action selection, this scalarization determines the greedy action:

$$a_t^* = \arg\min_{a\in\mathcal{A}} \psi_{\mathbf{w}}\left(\mathbf{Q}_\theta(s_t, a)\right). \quad (30)$$

The DDQN is trained with experience replay and a periodically updated target network [43]. At the start of each training episode, a preference vector is sampled as follows:

$$w_T \in \{0, 0.05, \ldots, 0.95, 1\}, \qquad w_E = 1 - w_T, \quad (31)$$

The sampled preference is included in the state and fixed throughout the episode, so the network observes different tardiness–energy trade-offs during training. Checkpoint validation evaluates each validation episode under all 21 grid preferences. For preference $\mathbf{w}$, the cumulative validation cost is $C^{\text{val}}(\mathbf{w}) = w_T \sum_t \Delta T_t + w_E \kappa \sum_t \Delta E_t$. The checkpoint with the lowest mean $C^{\text{val}}(\mathbf{w})$ across the validation episodes and grid preferences is retained. The remaining training parameters are reported in Table IV.

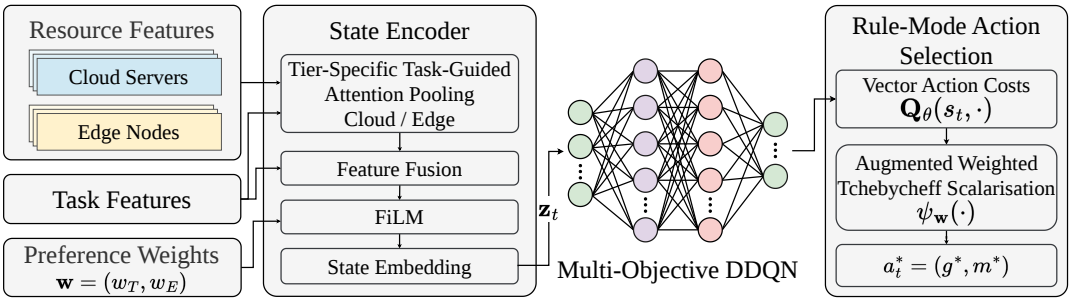


Fig. 2. Multi-objective DDQN for rule-mode action selection in PRS-GPDRL.

Algorithm 1: Online preference-aware resource selection.

Input: State s_t containing preference $\mathbf{w}$, ready task α_t, and feasible resource set $\mathcal{R}_t$
Output: Selected resource r_t^*

Encode s_t and evaluate $\mathbf{Q}_\theta(s_t, a)$ for all $a \in \mathcal{A}$;
Select $a_t^* = (g^*, m^*)$ by Eqs. (29) and (30);
Evaluate and normalize $g^*(r)$ over all $r \in \mathcal{R}_t$;
if $m^* = \text{base}$ **then** Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} \hat{g}^*(r)$;
Estimate the normalized costs $\hat{c}_T(r)$ and $\hat{c}_E(r)$ over $\mathcal{R}_t$;
Construct the auxiliary score $c_\mathbf{w}(r)$ and the four risk strata $\chi_\mathbf{w}(r)$ by Eqs. (21) and (22);
Compute $\beta_{\text{risk}}(\mathbf{w})$ by Eq. (23);
Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} p_{\text{risk}}(r; g^*)$ using Eq. (24);

F. Training and Inference Procedure

After CCGP training and action extraction in Section IV-B, the rule-mode action space is fixed for offline DRL training. A scenario is denoted by $\langle \text{wfNum}, \lambda \rangle$, where wfNum is the number of submitted workflows and λ is their arrival rate. Training and testing are performed separately for each scenario using the corresponding resource-selection rule pool and rule-mode action space. For the sampled preference, the DDQN learns from vector-cost transitions at decision points using the scalarization in Eq. (29). The trained DDQN and the fixed rule-mode action space constitute the PRS-GPDRL agent.

During inference, upward-rank ordering first determines the current ready task when several tasks are ready at the same decision point. The PRS-GPDRL agent then uses the DDQN to select a rule-mode action for this task. The selected action assigns α_t to the feasible resource with the minimum priority score, and the scheduling process proceeds to the next allocation decision point.

V. Performance Evaluation and Results

A. Baselines and Performance Measures

PRS-GPDRL is compared with CCGP, GATES, and LWFF.

- **CCGP** [21] coevolves task-selection, resource-selection, and container-scaling GP rules to construct non-dominated scheduling heuristics. For the problem model in Section III, our implementation retains its cooperative task- and resource-rule evolution but omits the container-scaling subpopulation because container CPU is assigned by Eq. (12). Each combined heuristic is evaluated on energy and weighted tardiness, and the final non-dominated heuristics form the approximation set.
- **GATES** [24] uses graph attention to encode ready tasks, workflow dependencies, and candidate resources, and evolves a neural resource-selection policy through an evolution strategy. Our implementation retains this graph representation and direct resource-selection policy. To support the two objectives, policy networks are evaluated on energy and weighted tardiness. The Non-dominated Sorting Genetic Algorithm II (NSGA-II) performs individual selection, and offspring networks are generated by Gaussian perturbations of the selected network parameters. The final non-dominated policies form the approximation set.
- **LWFF** [15] is a constructive heuristic that selects resources from non-dominated candidates according to resource waste and completion speed. In our implementation, ready tasks are ordered by upward rank, and the energy and tardiness estimates of feasible resources are min–max normalized and combined under each requested preference. Expected finish time resolves ties. Applying LWFF over the evaluation preference grid and retaining the non-dominated schedules produces its approximation set.

PRS-GPDRL uses the resource-selection rule pool extracted from the corresponding CCGP run. In each run, each method produces one non-dominated approximation set. PRS-GPDRL and LWFF use solutions over the preference grid, while CCGP and GATES use the final heuristics or policies. LWFF produces the same approximation set in every run. Its non-zero HV and IGD standard deviations arise because this set is jointly normalized with those of the other methods in each run. The four sets are pooled, and each objective $f_j \in \{\mathbb{T}, \mathbb{E}\}$ is min–max normalized as

$$\widehat{f}_j(x) = \frac{f_j(x) - f_j^{\min}}{f_j^{\max} - f_j^{\min}}, \tag{32}$$

where $f_j^{\min}$ and $f_j^{\max}$ are the pooled extrema of the four sets in that run. A constant objective is assigned zero.

HV evaluates the convergence and coverage of each approximation set and is computed against the reference point $(1, 1)$.

Because the true Pareto front is unknown, the IGD reference set Q comprises the jointly non-dominated normalized solutions from the four sets in that run. For approximation set P,

$$\mathrm{IGD}(P,Q) = \frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|p-q\|_2. \tag{33}$$

Larger HV and smaller IGD are preferable.

For each scenario and metric, the four algorithms are compared using the Iman–Davenport-corrected Friedman test and two-sided paired Wilcoxon signed-rank tests at a significance level of 0.05. Within each result cell, the arrows denote statistically better (↑), worse (↓), or similar (≈) performance against the preceding methods. For each baseline column, W/D/L (win/draw/loss) gives the number of scenarios in which that baseline is statistically better than, similar to, or worse than PRS-GPDRL. Therefore, the PRS-GPDRL column is marked as not applicable (N/A). AverageRank is the mean rank over the 30 runs, for which smaller is better.

B. Experimental Setup

Experiments use the event-driven workflow simulator introduced in [21]. Allocation updates occur at the task events specified in Section III. All methods use the same workflow instances, feasible resource set, communication model, CPU allocation rule, and energy accounting.

Table II lists five representative resource types for each tier. The configurations are derived from Amazon EC2[1], with CPU capacities reported in EC2 Compute Units (ECUs), and the power parameters follow the Teads model[2]. The Edge tier contains 19 fixed Edge Nodes, whereas Cloud Servers of the listed types can be provisioned on demand. Each task incurs a 50-ms container initialization delay [7], and provisioning a new Cloud Server takes 55.9 s. Following [45], [46], local-area network (LAN) and wide-area network (WAN) bandwidths are set to 2.0 and 0.5 GB/s, and their communication latencies are 0.5 and 30 ms, respectively.

Workflow templates are sampled from Alibaba Cluster-trace-v2018 [3]. The public trace supplies task dependencies and execution information, from which the DAG workflows used by the simulator are reconstructed. Task input and output data sizes follow a discrete uniform distribution between 500 and 5000 MB. Each task is independently assigned as CPU-sensitive or CPU-insensitive with equal probability. Workflow weights are independently sampled from $\{1, 2, 4\}$ with probabilities 0.2, 0.6, and 0.2, respectively, following [47].

Workflow arrivals follow a Poisson process with rate λ, so the inter-arrival time is exponentially distributed with mean $1/\lambda$ [48]. Following [21], reference execution and Edge transfer times are propagated along each DAG. Let L_ω be the largest earliest finish time obtained by this propagation. The workflow deadline is $\rho^d(\omega) = \rho^a(\omega) + \delta_\omega L_\omega$, where the deadline factor δ_ω is sampled uniformly from $\{0.8, 1.0, 1.5, 1.8\}$.

Training and test protocol.

Each scenario is denoted by $\langle \mathrm{wfNum}, \lambda\rangle$, where wfNum is the total number of submitted workflows and λ is their arrival rate. The main evaluation covers the 12 combinations of $\mathrm{wfNum} \in \{10, 15, 20, 30, 50, 100\}$ and $\lambda \in \{0.2, 0.8\}$. For each scenario, 30 independent runs are conducted. Within each run, every solution is evaluated on the same 30 unseen test instances, and its objective values are averaged over these instances before HV and IGD are calculated. The tables report the mean and standard deviation of HV and IGD over the 30 runs.

The parameter settings of CCGP follow [21] and are presented in Table III. The DDQN and risk-stratified action augmentation configuration is detailed in Table IV. GATES uses a population of 100 policy networks and runs for 51 generations.

We set $K = 5$. Behavioral filtering may leave fewer than five unique rules in a particular run. The actual pool size in run b is denoted by $K_b \le K$. Each retained rule is paired with the two modes. As a result, run b has $2K_b$ nominal rule-mode actions. Training preference sampling, checkpoint validation, and evaluation use the grid in Eq. (31).

C. Solution-Set Quality and Pareto Trade-offs

1) HV Results: Tables V and VI present the mean and standard deviation of the HV and IGD values, respectively, over the 30 independent runs of the compared algorithms.

PRS-GPDRL obtains the best HV average rank (1.406), while CCGP records 1/1/10 relative to it. HV gains are clearest at $\mathrm{wfNum} = 10$. PRS-GPDRL is statistically better than or comparable to CCGP throughout $\mathrm{wfNum} \le 50$. At $\mathrm{wfNum} = 100$, the outcome depends on the arrival rate: an additional gain occurs at $\langle 100, 0.2\rangle$, but not at $\langle 100, 0.8\rangle$, where the maximum workflow scale and higher arrival rate coincide.

2) IGD Results: PRS-GPDRL also obtains the best IGD average rank (1.478), with CCGP recording 1/4/7 relative to it. IGD improvements are again clearest at $\mathrm{wfNum} = 10$. PRS-GPDRL is statistically better than or comparable to CCGP throughout $\mathrm{wfNum} \le 50$, whereas the $\mathrm{wfNum} = 100$ outcomes depend on the arrival rate. The adapted GATES and LWFF baselines are statistically worse than or comparable to PRS-GPDRL in at least 11 scenarios per metric.

Because PRS-GPDRL extracts its resource-selection rule pool from the corresponding CCGP run, the comparison with CCGP controls the source of GP rules. Together with the behavior and ablation results, this pattern indicates that the improvement is more closely related to online selection among rules from the same CCGP run and bounded local adjustment than to a different source of GP rules.

3) 50% Empirical Attainment Curves: Fig. 3 presents the 50% empirical attainment curves of the compared algorithms in the normalized weighted-tardiness–energy space for six representative scenarios.

PRS-GPDRL lies below CCGP over most of their shared tardiness range in five panels, with the clearest separation in the two $\mathrm{wfNum} = 10$ scenarios. Because the separation spans the trade-off interior, the HV and IGD gains reflect broad set-quality improvements rather than a few endpoint solutions.

[1] https://aws.amazon.com/ec2/

[2] https://engineering.teads.com/sustainability/carbon-footprint-estimator-for-aws-instances/

[3] https://github.com/alibaba/clusterdata

TABLE II
CONFIGURATIONS OF EDGE NODES AND CLOUD SERVERS.

Edge Nodes					Cloud Servers				
Type (number)	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)	Type	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)
m1.small (3)	1	0.72	5.76	1.2	m4.xlarge	13	1.94	15.62	3.2
m3.medium (3)	3	0.87	6.97	1.4	m4.2xlarge	26	3.89	31.24	6.4
m3.large (4)	6.5	1.73	13.94	2.9	m4.4xlarge	53.5	7.77	62.47	12.9
m3.xlarge (4)	13	3.47	27.87	5.8	m4.10xlarge	124.5	24.12	193.88	40.0
m3.2xlarge (5)	26	6.94	55.74	11.5	m4.16xlarge	188	31.09	249.90	51.6

TABLE III
THE PARAMETER SETTINGS OF CCGP.

Parameter	Value
Population size	2 subpopulations of 50
Number of generations	51
Method for initializing population	Ramped-half-and-half
Initial minimum/maximum depth	2/7
Elitism	5
Maximal depth	8
Crossover/mutation/reproduction rate	0.80/0.15/0.05
Terminal/non-terminal selection rate	10%/90%
Environmental selection	NSGA-II

TABLE IV
THE PARAMETER CONFIGURATION OF DDQN AND RISK-STRATIFIED ACTION AUGMENTATION.

Parameter	Value
Training-step budget	1,000,000
Exploration rate ϵ	1.0 decays to 0.01
Discount factor γ	1.0
Learning rate	5×10^{-5}
Optimizer	Adam
Mini-batch size	256
Replay memory size	200,000
Target-update frequency	5,000
DDQN hidden-layer sizes	256/128
Maximum resource-selection rule pool size K	5
Maximum risk mixing $\beta_{\max}$	0.75
Energy-cost scaling κ	4.7×10^5
Tchebycheff augmentation ζ	0.05

TABLE V
THE MEAN AND STANDARD DEVIATION OF THE HV VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle$wfNum, $\lambda\rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2\rangle$	0.533±0.189	0.597±0.131(≈)	0.789±0.092(↑)(↑)	0.867±0.068(↑)(↑)(↑)
$\langle 10, 0.8\rangle$	0.536±0.211	0.616±0.124(↑)	0.788±0.113(↑)(↑)	0.882±0.059(↑)(↑)(↑)
$\langle 15, 0.2\rangle$	0.566±0.124	0.602±0.107(≈)	0.800±0.114(↑)(↑)	0.862±0.087(↑)(↑)(↑)
$\langle 15, 0.8\rangle$	0.539±0.128	0.651±0.109(↑)	0.749±0.150(↑)(↑)	0.843±0.081(↑)(↑)(↑)
$\langle 20, 0.2\rangle$	0.607±0.168	0.632±0.122(≈)	0.789±0.134(↑)(↑)	0.869±0.088(↑)(↑)(↑)
$\langle 20, 0.8\rangle$	0.530±0.170	0.646±0.107(↑)	0.776±0.110(↑)(↑)	0.825±0.060(↑)(↑)(↑)
$\langle 30, 0.2\rangle$	0.635±0.139	0.675±0.091(↑)	0.823±0.101(↑)(↑)	0.867±0.060(↑)(↑)(↑)
$\langle 30, 0.8\rangle$	0.568±0.132	0.697±0.091(↑)	0.788±0.091(↑)(↑)	0.803±0.090(↑)(↑)(≈)
$\langle 50, 0.2\rangle$	0.611±0.152	0.702±0.098(↑)	0.808±0.103(↑)(↑)	0.851±0.114(↑)(↑)(↑)
$\langle 50, 0.8\rangle$	0.644±0.164	0.749±0.103(↑)	0.783±0.135(↑)(↑)	0.815±0.110(↑)(↑)(↑)
$\langle 100, 0.2\rangle$	0.579±0.192	0.697±0.090(↑)	0.817±0.096(↑)(↑)	0.847±0.077(↑)(↑)(↑)
$\langle 100, 0.8\rangle$	0.798±0.081	0.866±0.059(↑)	0.852±0.067(↑)(↓)	0.810±0.086(≈)(↓)(↓)
W/D/L	0/1/11	1/0/11	1/1/10	N/A
AverageRank	3.603	3.017	1.975	1.406

TABLE VI
THE MEAN AND STANDARD DEVIATION OF THE IGD VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle$wfNum, $\lambda\rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2\rangle$	0.321±0.135	0.235±0.081(↑)	0.100±0.032(↑)(↑)	0.047±0.024(↑)(↑)(↑)
$\langle 10, 0.8\rangle$	0.310±0.145	0.199±0.063(↑)	0.107±0.038(↑)(↑)	0.042±0.020(↑)(↑)(↑)
$\langle 15, 0.2\rangle$	0.257±0.099	0.166±0.054(↑)	0.068±0.031(↑)(↑)	0.040±0.028(↑)(↑)(↑)
$\langle 15, 0.8\rangle$	0.303±0.094	0.154±0.054(↑)	0.084±0.057(↑)(↑)	0.047±0.034(↑)(↑)(↑)
$\langle 20, 0.2\rangle$	0.260±0.128	0.173±0.077(↑)	0.078±0.041(↑)(↑)	0.050±0.048(↑)(↑)(↑)
$\langle 20, 0.8\rangle$	0.276±0.133	0.151±0.075(↑)	0.074±0.035(↑)(↑)	0.055±0.038(↑)(↑)(≈)
$\langle 30, 0.2\rangle$	0.247±0.110	0.152±0.050(↑)	0.083±0.037(↑)(↑)	0.053±0.046(↑)(↑)(↑)
$\langle 30, 0.8\rangle$	0.291±0.108	0.127±0.039(↑)	0.080±0.030(↑)(↑)	0.074±0.043(↑)(↑)(≈)
$\langle 50, 0.2\rangle$	0.232±0.095	0.112±0.040(↑)	0.082±0.042(↑)(↑)	0.059±0.043(↑)(↑)(↑)
$\langle 50, 0.8\rangle$	0.240±0.105	0.099±0.032(↑)	0.074±0.039(↑)(↑)	0.075±0.041(↑)(↑)(≈)
$\langle 100, 0.2\rangle$	0.269±0.139	0.146±0.063(↑)	0.075±0.034(↑)(↑)	0.079±0.040(↑)(↑)(≈)
$\langle 100, 0.8\rangle$	0.190±0.078	0.079±0.055(↑)	0.066±0.024(↑)(≈)	0.135±0.056(↑)(↓)(↓)
W/D/L	0/0/12	1/0/11	1/4/7	N/A
AverageRank	3.792	2.847	1.883	1.478

D. Preference-Aware Policy Behavior

1) Objective Response to Preference: Fig. 4 shows the scenario-wise median relative reductions in weighted tardiness and energy consumption as their corresponding preference weights vary across the 12 scenarios. Let $x(w_j)$ denote the solution obtained at objective weight w_j. For objective f_j, the relative reduction is $[f_j(x(0)) - f_j(x(w_j))]/f_j(x(0))$ from the endpoint at which its own weight is zero. Weighted tardiness is plotted against w_T, and energy against $w_E = 1 - w_T$.

At the outcome level, increasing an objective's weight changes the schedule in the intended direction. Median reductions at weight 0.5 are 89.7% for weighted tardiness and 20.5% for energy. Over the full range, they reach 96.9% and 51.1%. The steeper tardiness response is consistent with urgency-oriented decisions avoiding deadline penalties immediately, whereas energy savings accumulate through assignments that alter utilization and resource active time.

The median correlations between each raw objective and its own weight are $\rho_T = -0.968$ and $\rho_E = -0.955$. All tardiness curves and 99.4% of energy curves have the expected direction. In addition, 99.4% of runs shift correctly between the endpoints. Thus, one network produces preference-dependent scheduling outcomes rather than relabeling the same outcome with different weights.

2) Preference-Dependent Resource-Selection Rule Usage: Fig. 5 shows how the use of the five retained resource-selection rules varies with tardiness preference across the 12 scenarios. The rules are ordered from the minimum-energy endpoint to the minimum-tardiness endpoint.

At the rule level, the minimum-energy rule and its neighbor account for 61.7% of decisions at $w_T = 0$, while the minimum-tardiness pair accounts for 50.3% at $w_T = 1$. The median weight–role correlation is $\rho_g = 0.738$, and 90.4% of included runs shift as expected. This explains Fig. 4 internally:

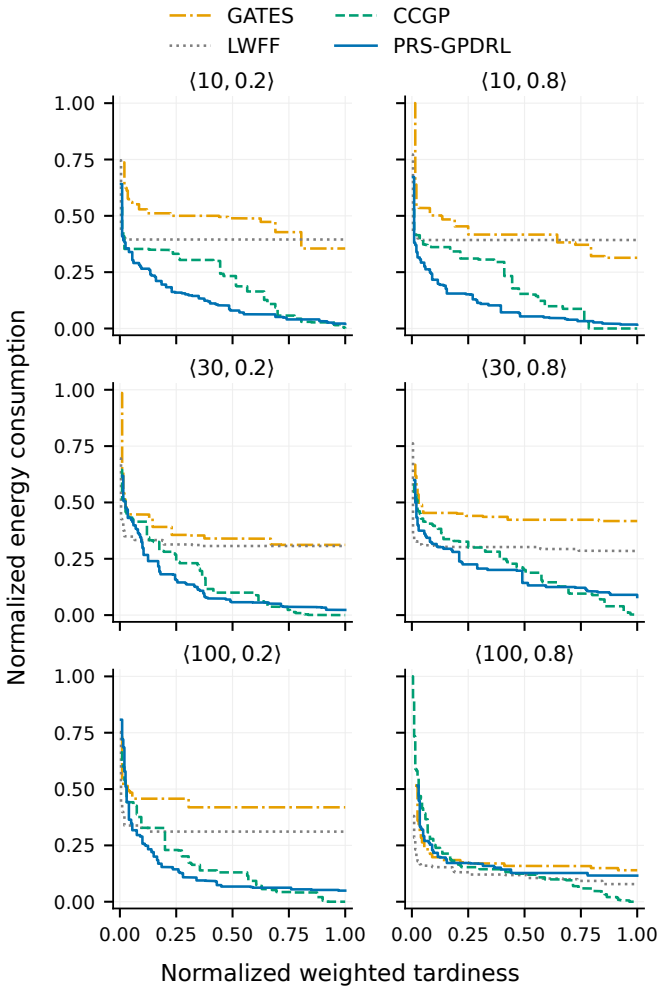


Fig. 3. The 50% empirical attainment curves of the compared algorithms in six representative scenarios.

increasing w_T moves selection from energy-oriented roles through the knee toward tardiness-oriented roles.

3) Risk-Mode Usage: Fig. 6 shows the risk-mode selection frequency as a function of w_T for the two arrival rates, with one panel for each workflow count.

At the mode level, the risk mode is selected in 42.5% of decisions. Away from balance, non-zero correction is activated in 41.2%, ranging from 26.1% to 56.6% across preferences and scenarios. These values indicate that the risk mode is used in a substantial fraction of decisions. At the same time, the remaining base-mode decisions indicate that local correction is not always required. The policy adaptively chooses between using the unmodified GP ranking and applying a local correction as the state and preference vary.

Figs. 5 and 6 capture two distinct levels of adaptation. Rule selection sets the global objective direction of the resource ranking by moving from energy-oriented roles through the knee toward tardiness-oriented roles as w_T increases. Risk-mode selection instead determines whether the current decision requires a bounded local modification of that ranking. Its frequency need not vary monotonically with w_T, because the need for correction depends jointly on the requested preference, task urgency, CPU sharing, resource active time, Cloud provisioning, and the current feasible-resource state. Thus, risk-mode frequency reflects the conditional use of local correction under the current state and preference rather than serving as a direct proxy for the preference weight.

Within the evaluated preference grid, the three figures form a layered account of policy behavior. Fig. 4 shows preference response at the outcome level. Fig. 5 shows the corresponding migration of global rule roles. Fig. 6 shows whether local correction is selected at the mode level. Taken together, these observations support the interpretation that preference conditioning changes both the resulting objectives and the internal decisions that produce them: which global ranking is used and whether it is locally corrected for the current state.

E. Ablation Analysis

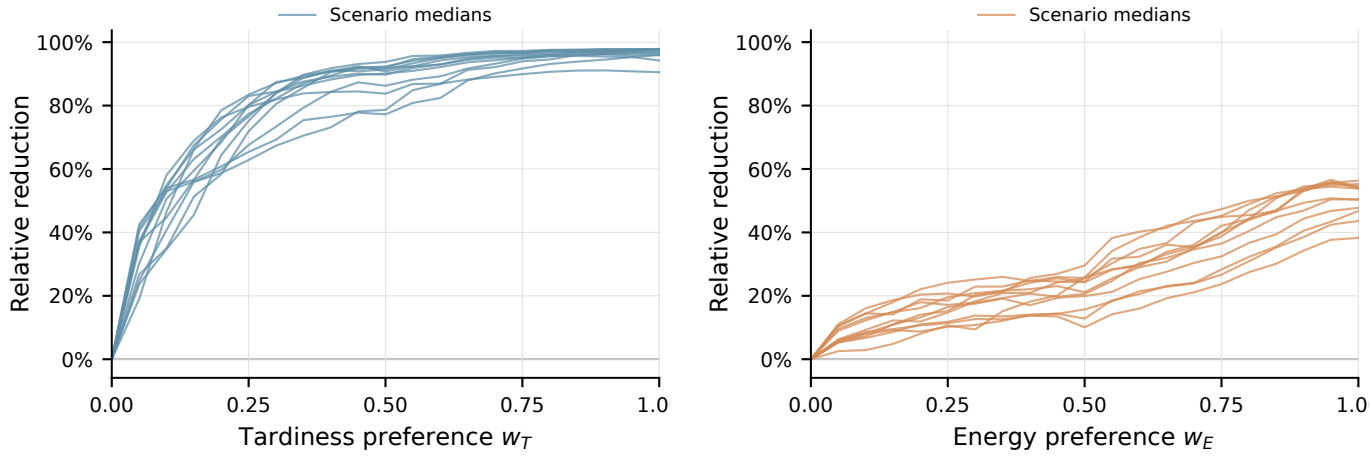
Table VII reports the HV and IGD results of PRS-GPDRL and its three ablation variants in six representative scenarios. The *w/o Risk* and *w/o FiLM* variants remove the two components separately, while $K = 3$ sets the maximum requested resource-selection rule pool size to three. The evaluation and statistical analysis follow the main protocol.

1) Risk-Stratified Action Augmentation and FiLM Conditioning: Across the six representative scenarios, *w/o Risk* records 0/0/6 against PRS-GPDRL for both metrics, the strongest ablation effect. A GP rule provides a global ranking. Risk uses the specified preference, task urgency, CPU sharing, resource active time, and Cloud provisioning for bounded local correction. The consistent result across these six scenarios supports state- and preference-dependent local correction as an important part of the full performance improvement.

By comparison, the *w/o FiLM* variant records 0/3/3 for both metrics and retains the vector-valued action costs and augmented weighted Tchebycheff scalarization, so preference information remains available to action evaluation. The three statistically comparable outcomes and three losses suggest that FiLM is not the only way in which preference information enters the policy. Its additional preference-dependent modulation of the shared state representation is consistent with complementary improvements in trade-off coverage and proximity.

2) Rule-Pool Size: The $K = 3$ result is 0/5/1 for both metrics and is statistically comparable in five of six scenarios. This indicates that the main capability does not arise simply from increasing the number of available actions. A small set of behaviorally complementary rules retains most of it. Within the tested settings, performance is reasonably robust to pool size when behaviorally diverse rule roles are retained. The $K = 5$ design targets both endpoints, their neighboring regions, and the knee, and obtains the best average ranks. The comparison supports the motivation for quality- and behavior-aware filtering: coverage of complementary global ranking roles is more important than rule count alone.

The ablations thus organize the components into distinct, complementary roles: Risk provides state- and preference-dependent local correction, FiLM adds preference-dependent modulation of the shared representation, and the resource-selection rule pool provides complementary global ranking



(a) Weighted tardiness response to w_T.

(b) Energy response to w_E.

Fig. 4. Relative reductions in weighted tardiness and energy consumption under different preference weights. Each curve shows the median over 30 independent runs for one scenario.

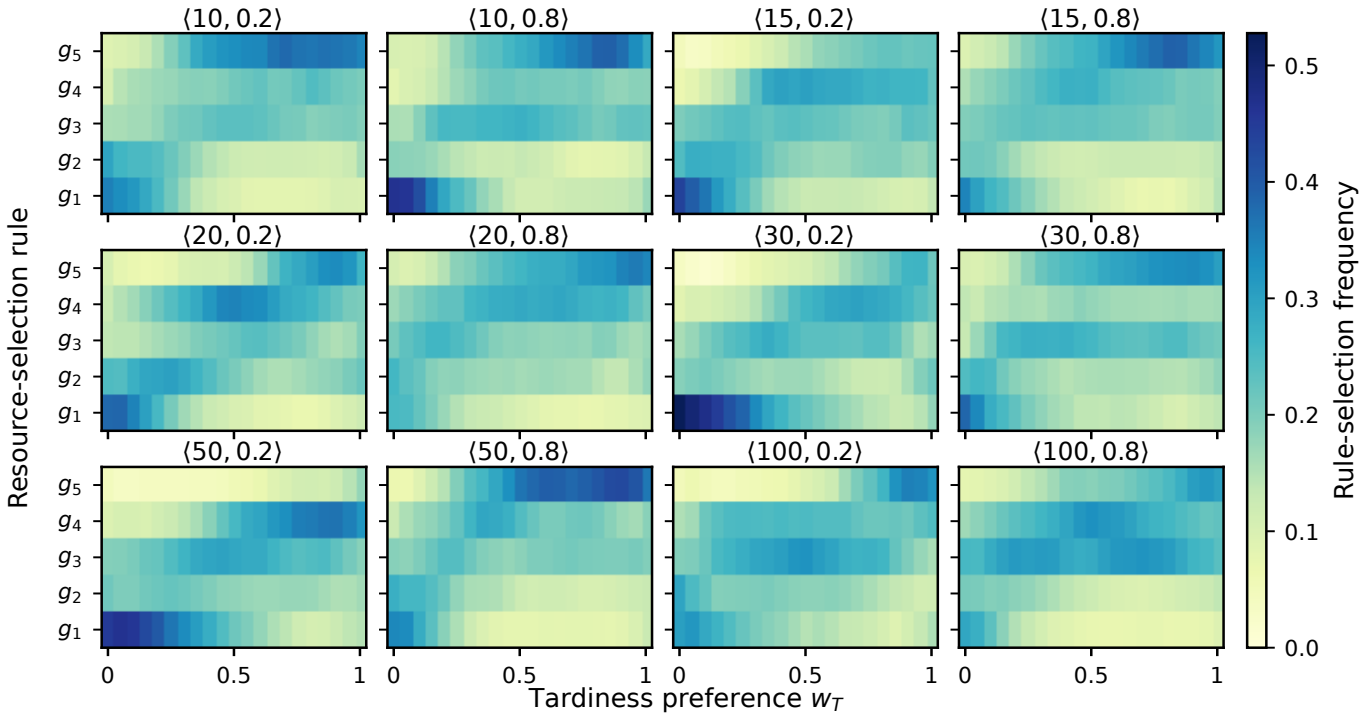


Fig. 5. Mean selection frequencies over the included independent runs for each of the 12 scenarios under different tardiness preferences (runs retaining five rules). The roles are the minimum-energy endpoint g_1, its neighbor g_2, the knee g_3, the minimum-tardiness neighbor g_4, and the minimum-tardiness endpoint g_5.

coverage. These roles are complementary within the proposed decision process.

F. Task-Ordering Design Choice

Table VIII reports the HV and IGD results obtained using the GP task-selection rule and upward-rank ordering across the 12 scenarios. The comparison holds the deduplicated resource-selection rule pool fixed. Following Tables V–VII, W/D/L reports the baseline GP task-selection rule relative to upward-rank ordering.

With the resource-selection rule pool fixed, the GP task-selection rule records 1/11/0 against upward-rank ordering for both metrics, with similar average ranks. Thus, upward-rank ordering is statistically comparable in 11 of 12 scenarios. The diagnostic result indicates that task-ordering change is not the main explanation for the overall gain over CCGP. The comparability is consistent with the problem structure: VMs can execute tasks concurrently, and task ordering is required only when multiple tasks become ready simultaneously, whereas resource selection is required for every dispatched task. The

TABLE VII
THE MEAN AND STANDARD DEVIATION OF THE HV AND IGD VALUES FOR PRS-GPDRL AND ITS ABLATION VARIANTS OVER 30 INDEPENDENT RUNS.

$\langle \text{wfNum}, \lambda \rangle$	HV				IGD			
	w/o Risk	w/o FiLM	$K = 3$	PRS-GPDRL	w/o Risk	w/o FiLM	$K = 3$	PRS-GPDRL
$\langle 10, 0.2 \rangle$	.832±.098	.851±.076(≈)	.860±.071(↑)(↑)	.867±.068(↑)(↑)(≈)	.078±.046	.058±.030(↑)	.052±.033(↑)(≈)	.047±.024(↑)(↑)(≈)
$\langle 10, 0.8 \rangle$	.827±.093	.855±.068(↑)	.880±.064(↑)(↑)	.882±.059(↑)(↑)(≈)	.076±.047	.058±.032(↑)	.039±.017(↑)(↑)	.042±.020(↑)(↑)(≈)
$\langle 30, 0.2 \rangle$	.814±.103	.848±.074(↑)	.851±.076(≈)(≈)	.867±.060(↑)(↑)(↑)	.090±.058	.068±.047(↑)	.068±.057(↑)(≈)	.053±.046(↑)(↑)(↑)
$\langle 30, 0.8 \rangle$	.752±.105	.814±.087(↑)	.809±.081(↑)(≈)	.803±.090(↑)(≈)(≈)	.114±.059	.065±.032(↑)	.074±.034(↑)(≈)	.074±.043(↑)(≈)(≈)
$\langle 100, 0.2 \rangle$	.778±.117	.828±.075(↑)	.854±.055(↑)(↑)	.847±.077(↑)(≈)(≈)	.118±.071	.084±.047(↑)	.082±.042(↑)(≈)	.079±.040(↑)(≈)(≈)
$\langle 100, 0.8 \rangle$	.748±.108	.801±.090(↑)	.816±.081(↑)(≈)	.810±.086(↑)(≈)(≈)	.176±.065	.125±.052(↑)	.121±.051(↑)(≈)	.135±.056(↑)(≈)(≈)
W/D/L	0/0/6	0/3/3	0/5/1	N/A	0/0/6	0/3/3	0/5/1	N/A
AverageRank	3.222	2.511	2.164	2.103	3.322	2.389	2.231	2.058

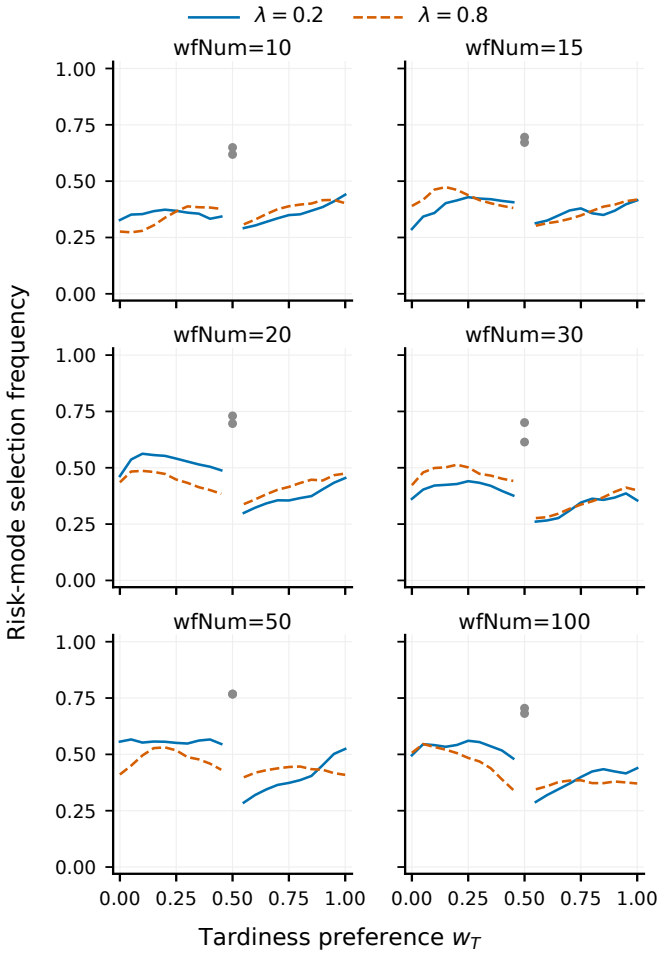


Fig. 6. Mean risk-mode selection frequency over 30 independent runs for each of the 12 scenarios. Gray markers denote the balanced preference $w_T = w_E = 0.5$. At the balanced preference, risk-mode selections are counted although their priority scores coincide with those of the base mode.

comparison suggests that task ordering is less influential than resource selection in this problem.

Taken together, the experiments help explain the overall gain. The comparison with CCGP controls the source of GP rules because PRS-GPDRL uses the resource-selection rule pool extracted from the corresponding CCGP run. The task-ordering comparison indicates that replacing the GP task-selection rule with upward-rank ordering is not the main factor.

TABLE VIII
THE MEAN AND STANDARD DEVIATION OF THE HV AND IGD VALUES OBTAINED USING GP TASK-SELECTION RULE AND UPWARD-RANK ORDERING OVER 30 INDEPENDENT RUNS.

$\langle \text{wfNum}, \lambda \rangle$	HV		IGD	
	GP task rule	Upward rank	GP task rule	Upward rank
$\langle 10, 0.2 \rangle$	.789±.092	.790±.092(≈)	.100±.032	.100±.033(≈)
$\langle 10, 0.8 \rangle$	.788±.113	.787±.117(≈)	.107±.038	.108±.042(≈)
$\langle 15, 0.2 \rangle$	.800±.114	.799±.109(≈)	.068±.031	.070±.028(≈)
$\langle 15, 0.8 \rangle$	.749±.150	.750±.151(≈)	.084±.057	.085±.059(≈)
$\langle 20, 0.2 \rangle$	.789±.134	.787±.136(≈)	.078±.041	.078±.040(≈)
$\langle 20, 0.8 \rangle$	.776±.110	.779±.108(≈)	.074±.035	.072±.033(≈)
$\langle 30, 0.2 \rangle$	.823±.101	.823±.100(≈)	.083±.037	.081±.037(≈)
$\langle 30, 0.8 \rangle$	.788±.091	.784±.099(≈)	.080±.030	.081±.030(≈)
$\langle 50, 0.2 \rangle$	.808±.103	.806±.104(≈)	.082±.042	.084±.042(↓)
$\langle 50, 0.8 \rangle$	.783±.135	.779±.136(↓)	.074±.039	.075±.037(≈)
$\langle 100, 0.2 \rangle$	.817±.096	.813±.096(≈)	.075±.034	.078±.035(≈)
$\langle 100, 0.8 \rangle$	.852±.067	.850±.070(≈)	.066±.024	.067±.026(≈)
W/D/L	1/11/0	N/A	1/11/0	N/A
AverageRank	1.436	1.564	1.458	1.542

The rule-usage analysis shows preference-dependent use of rules extracted from the same CCGP run, and the mode-usage analysis shows conditional rather than uniform local correction. The $K = 3$ comparison further weakens a simple action-count explanation, while the *w/o Risk* result supports the importance of local correction for the full improvement in the six ablation scenarios. These results jointly support preference-aware online use of behaviorally complementary GP rules, together with bounded local correction when indicated by the current state.

VI. CONCLUSION

This paper studies preference-aware online scheduling of dynamic workflows in heterogeneous Cloud–Edge systems, with total weighted tardiness and energy as the two objectives. PRS-GPDRL connects evolutionary rule discovery with online reinforcement learning. CCGP evolves coupled scheduling rules, a diversity-aware procedure extracts a compact resource-selection rule pool, and a multi-objective DDQN selects risk-stratified rule-mode actions at allocation decision points. Through this process, one network can adapt the use of GP-evolved resource priorities to the current state and requested objective trade-off.

Across 12 scenarios, with 30 independent runs conducted per scenario, PRS-GPDRL obtains the best average ranks of 1.406 for HV and 1.478 for IGD and is statistically better

than or comparable to CCGP in 11 of 12 scenarios for each metric. The clearest advantages occur on small- and medium-scale workflow sets, with additional HV gains in selected large-scale scenarios. The behavior analysis shows preference response at three levels: the objective values, the roles of the selected resource-selection rules, and the choice between using an unmodified GP ranking and applying a local risk correction. Across the six representative ablation scenarios, Risk provides the strongest component-level improvement, FiLM adds complementary preference conditioning, and the $K = 3$ results retain most of the resource-selection rule pool capability. The task-ordering ablation experiment further shows that upward-rank ordering is statistically comparable to the GP task-selection rule in 11 of the 12 scenarios. Overall, PRS-GPDRL generates high-quality tardiness–energy trade-offs while adapting its internal scheduling decisions to the requested preference.

REFERENCES

[1] S. Li, W. Wu, H. Zhang, Y. Liu, W. Lin, and K. Li, "Revisiting workflow scheduling with the power of edge computing: Taxonomy, review, and open challenges," *Comput. Sci. Rev.*, vol. 60, p. 100887, 2026.

[2] R. Bouabdallah and F. Fakhfakh, "Workflow scheduling in cloud–fog computing environments: A systematic literature review," *Concurr. Comput. Pract. Exp.*, vol. 36, no. 28, p. e8304, 2024.

[3] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022.

[4] Gartner, "Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025," 2025, accessed: Jul. 27, 2026. [Online]. Available: https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025

[5] International Energy Agency, "Energy and AI," 2025, accessed: Jul. 27, 2026. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[6] Central Statistics Office, "Data centres metered electricity consumption 2025," 2026, accessed: Aug. 18, 2026. [Online]. Available: https://www.cso.ie/en/releasesandpublications/ep/p-dcmec/datacentresmeteredelectricityconsumption2025/

[7] R. Ranjan, I. S. Thakur, G. S. Aujla, N. Kumar, and A. Y. Zomaya, "Energy-efficient workflow scheduling using container-based virtualization in software-defined data centers," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7646–7657, 2020.

[8] Kubernetes, "Resize CPU and memory resources assigned to containers," 2026, accessed: Jul. 27, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/

[9] Docker Inc., "docker container update," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/update/

[10] ——, "Resource constraints," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/

[11] G. Fan, X. Chen, Z. Li, H. Yu, and Y. Zhang, "An energy-efficient dynamic scheduling method of deadline-constrained workflows in a cloud environment," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3089–3103, 2023.

[12] Z. Sun, H. Huang, Z. Li, and C. Gu, "Energy-efficient real-time multi-workflow scheduling in container-based cloud," *J. Comb. Optim.*, vol. 49, no. 2, pp. 1–21, 2025.

[13] K. Li, "Energy-constrained DAG scheduling on edge and cloud servers with overlapped communication and computation," *J. Grid Comput.*, vol. 22, no. 3, p. 60, 2024.

[14] J. Lou, Z. Tang, S. Zhang, W. Jia, W. Zhao, and J. Li, "Cost-effective scheduling for dependent tasks with tight deadline constraints in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5829–5845, 2023.

[15] H. M. Fard, R. Prodan, and F. Wolf, "Dynamic multi-objective scheduling of microservices in the cloud," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput.*, 2020, pp. 386–393.

[16] K.-R. Escott, H. Ma, and G. Chen, "A genetic programming hyper-heuristic approach to design high-level heuristics for dynamic workflow scheduling in cloud," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2020, pp. 3141–3148.

[17] Y. Yu, T. Shi, H. Ma, and G. Chen, "A genetic programming-based hyper-heuristic approach for multi-objective dynamic workflow scheduling in cloud environment," in *Proc. IEEE Congr. Evol. Comput.*, 2022, pp. 1–8.

[18] M. Xu, Y. Mei, S. Zhu, B. Zhang, T. Xiang, F. Zhang, and M. Zhang, "Genetic programming for dynamic workflow scheduling in fog computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2657–2671, 2023.

[19] Z. Sun, Y. Mei, F. Zhang, H. Huang, C. Gu, and M. Zhang, "Multi-tree genetic programming hyper-heuristic for dynamic flexible workflow scheduling in multi-clouds," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2687–2703, 2024.

[20] Y. Yang, G. Chen, H. Ma, S. Hartmann, and M. Zhang, "Dual-tree genetic programming with adaptive mutation for dynamic workflow scheduling in cloud computing," *IEEE Trans. Evol. Comput.*, vol. 29, no. 5, pp. 1692–1706, 2025.

[21] Z. Sun, F. Zhang, Y. Mei, H. Huang, C. Gu, B. Wang, and M. Zhang, "Cooperative coevolution genetic programming for dynamic joint workflow scheduling and container scaling in cloud-fog computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 225–239, 2026.

[22] A. Jayanetti, S. Halgamuge, and R. Buyya, "Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments," *Future Gener. Comput. Syst.*, vol. 137, pp. 14–30, 2022.

[23] B. Xiao, C. Yu, X. Chen, Z. Chen, and G. Min, "Multi-agent collaboration for workflow task offloading in end-edge-cloud environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 11, pp. 2281–2296, 2025.

[24] Y. Shen, G. Chen, H. Ma, and M. Zhang, "GATES: Cost-aware dynamic workflow scheduling via graph attention networks and evolution strategy," in *Proc. 34th Int. Joint Conf. Artif. Intell.*, 2025, pp. 8635–8643.

[25] R. Liu, R. Piplani, and C. Toro, "Deep reinforcement learning for dynamic scheduling of a flexible job shop," *Int. J. Prod. Res.*, vol. 60, no. 13, pp. 4049–4069, 2022.

[26] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Niching genetic programming to learn actions for deep reinforcement learning in dynamic flexible scheduling," *IEEE Trans. Evol. Comput.*, vol. 30, no. 1, pp. 61–75, 2026.

[27] F. Zhao, C. Zhao, L. Wang, and H. Sang, "A deep reinforcement learning framework assisted by genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, 2025.

[28] Y. He, Z. Xu, J. Lin, Y. Li, and S. Zhang, "Deep reinforcement learning with evolved actions for dynamic workflow scheduling in distributed fog computing," *Neurocomputing*, vol. 677, p. 133115, 2026.

[29] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, "A survey of multi-objective sequential decision-making," *J. Artif. Intell. Res.*, vol. 48, pp. 67–113, 2013.

[30] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers, "A practical guide to multi-objective reinforcement learning and planning," *Auton. Agents Multi-Agent Syst.*, vol. 36, no. 1, p. 26, 2022.

[31] A. Abels, D. M. Roijers, T. Lenaerts, A. Nowé, and D. Steckelmacher, "Dynamic weights in multi-objective deep reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 11–20.

[32] R. Yang, X. Sun, and K. Narasimhan, "A generalized algorithm for multi-objective reinforcement learning and policy adaptation," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[33] X. Lin, Z. Yang, X. Zhang, and Q. Zhang, "Pareto set learning for expensive multi-objective optimization," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 19 231–19 247.

[34] M. Xu, Y. Mei, F. Zhang, Y. S. Ong, and M. Zhang, "Pareto set learning through genetic programming for multiobjective dynamic scheduling," *IEEE Trans. Evol. Comput.*, vol. 30, no. 3, pp. 942–956, 2026.

[35] J. Zhang, Z. Ning, M. Waqas, H. Alasmary, S. Tu, and S. Chen, "Hybrid edge-cloud collaborator resource scheduling approach based on deep reinforcement learning and multiobjective optimization," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 192–205, 2024.

[36] Y. Li, Y. He, J. Lin, Z. Xu, and S. Zhang, "A reinforcement learning-based population hyper-heuristic for energy-efficient cloud workflow scheduling problem," *IEEE Trans. Serv. Comput.*, vol. 18, no. 5, pp. 2545–2558, 2025.

[37] K. Van Moffaert and A. Nowé, "Multi-objective reinforcement learning using sets of Pareto dominating policies," *J. Mach. Learn. Res.*, vol. 15, pp. 3663–3692, 2014.
[38] S. Parisi, M. Pirotta, and M. Restelli, "Multi-objective reinforcement learning through continuous Pareto manifold approximation," *J. Artif. Intell. Res.*, vol. 57, pp. 187–227, 2016.
[39] M. Reymond, E. Bargiacchi, and A. Nowé, "Pareto conditioned networks," in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst.*, 2022, pp. 1110–1118.
[40] X. Wu, Q. Zhu, Q. Lin, K.-C. Wong, W.-N. Chen, and C. A. Coello Coello, "Experience evolution-guided multi-objective reinforcement learning," *IEEE Trans. Evol. Comput.*, 2026.
[41] Y. Yang, T. Zhou, M. Pechenizkiy, and M. Fang, "Preference controllable reinforcement learning with advanced multi-objective optimization," in *Proc. 42nd Int. Conf. Mach. Learn.*, vol. 267, 2025, pp. 71 612–71 647.
[42] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
[43] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[44] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3942–3951.
[45] M. Goudarzi, H. Wu, M. Palaniswami, and R. Buyya, "An application placement technique for concurrent IoT applications in edge and fog computing environments," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1298–1311, 2021.
[46] S. Pallewatta, V. Kostakos, and R. Buyya, "QoS-aware placement of microservices-based IoT applications in fog computing environments," *Future Gener. Comput. Syst.*, vol. 131, pp. 121–136, 2022.
[47] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.
[48] S. Sahoo, B. Sahoo, and A. K. Turuk, "A learning automata-based scheduling for deadline sensitive task in the cloud," *IEEE Trans. Serv. Comput.*, vol. 14, no. 6, pp. 1662–1674, 2021.